11
Generici e collezioni, pt 2

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2020/2021

Outline

Goal della lezione

- Approfondire alcuni concetti sui generici
- Presentare altre classi per le collezioni

Argomenti

- Il problema della type-erasure
- Polimorfismo vincolato
- Approfondimento sulle Wildcards
- Implementazioni di `List` e `Map`

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

La classe generica List

```
/* Classe generica in X:
   - X è il tipo degli elementi della lista */
public class List<X>{

    private final X head;     // Testa della lista, tipo X
    private final List<X> tail;  // Coda della lista, tipo List<X>

    public List(final X head, final List<X> tail){
      this.head = head;
      this.tail = tail;
    }

    public X getHead(){
      return this.head;
    }

    public List<X> getTail(){
      return this.tail;
    }

    // getLength() e toString() invariate
    ...
}
```

Uso di una classe generica

```
public class UseList{
    public static void main(String[] s){
      List<Integer> list = new List<Integer>(10, // Autoboxing
                    new List<Integer>(20,
                    new List<Integer>(30,
                    new List<Integer>(40,null))));
      // Cast NON necessari
      int first = list.getHead(); // Unboxing
      int second = list.getTail().getHead();
      int third = list.getTail().getTail().getHead();
      System.out.println(first+" "+second+" "+third);
      System.out.println(list.toString());
      System.out.println(list.getLength());

      // Usabile anche con le stringhe
      List<String> list2 = new List<String>("a",
                    new List<String>("b",
                    new List<String>("c",
                    new List<String>("d",null))));
      System.out.println(list2.toString());
    }
}
```

Terminologia, e elementi essenziali

Data una classe generica `C<X,Y>`..

- `X` e `Y` sono dette le sue type-variable
- `X` e `Y` possono essere usate come un qualunque tipo dentro la classe (con alcune limitazioni che vedremo)

I clienti delle classi generiche

- Devono usare tipi generici: versioni "istanziate" delle classi generiche
  - `C<String,Integer>`, `C<C<Object,Object>,Object>`
  - Non `C` senza parametri, altrimenti vengono segnalati dei warning
- Ogni type-variable va sostituita con un tipo effettivo, ossia con un parametro, che può essere
  - una classe (non-generica), p.e. `Object`, `String`,..
  - una type-variable definita, p.e. `X,Y` (usate dentro la classe `C<X,Y>`)
  - un tipo generico completamente istanziato, p.e. `C<Object,Object>`
  - ..o parzialmente istanziato, p.e. `C<Object,X>` (in `C<X,Y>`)
  - NON con un tipo primitivo

Limitazioni all'uso dei generici

Una type-variable (`X`) non è usabile in istruzioni del tipo:

- `new X()`, `new X[]{..}`, `new X[10]`, `instanceof X`
- Il compilatore segnala un errore
- errore anche l'`instanceof` su un tipo generico:
  `o instanceof C<String>`

Type-variable, e tipi generici danno warning se usati in situazioni "borderline"

- ... `(X)o`, `(C<String>)o`
- il compilatore segnala un "unchecked warning"

Perché queste limitazioni? (Odersky & Runne & Wadler, 1998)

- Derivano dallo schema di supporto ad erasure
- Il compilatore prende la classe `List<X>` e la trasforma in qualcosa di simile alla classe `ObjectList` prima di compilarla effettivamente..

Qualche prova

```
import java.util.ArrayList;
import java.util.List;

class P<X> {
  void m() {
    X x = (X)new Object(); // warning
    // X x = new X(); // error
    X[] a = (X[])new Object[10]; // warning
    // X[] a = new X[10]; // error
    // boolean b = new Object() instanceof X; // error
  }
}

public class ErasurePitfalls {

  public static void main(String[] args) {
    Object o = (List<String>)new Object();
    // boolean b = new Object() instanceof List<String> // error
    // Object o = new List<Pair>[10]; // error
  }

}
```

La classe generica Vector

Un dettaglio della sua implementazione:

Per via della type erasure, il suo campo non può essere di tipo `X[]`, bensì `Object[]`

```
public class Vector<X>{
// X è la type-variable, ossia il tipo degli elementi
    ...
    public Vector(){...}

    public void addElement(X e){...}  // Input di tipo X

    public X getElementAt(int pos){...} // Output di tipo X

    public int getLength(){...}

    public String toString(){...}
}
```

Uso di Vector<X>

```
public class UseVector{
  public static void main(String[] s){

      // Il tipo di vs è Vector<String>
      // Ma la sua classe è Vector<X>
      final Vector<String> vs = new Vector<String>();
      vs.addElement("Prova");
      vs.addElement("di");
      vs.addElement("Vettore");
      final String str = vs.getElementAt(0) + " " +
             vs.getElementAt(1) + " " +
             vs.getElementAt(2); // Nota, nessun cast!
      System.out.println(str);

      final Vector<Integer> vi=new Vector<Integer>();
      vi.addElement(10); // Autoboxing
      vi.addElement(20);
      vi.addElement(30);
      final int i = vi.getElementAt(0) + // Unboxing
        vi.getElementAt(1) +
        vi.getElementAt(2);
      System.out.println(i);
    }
}
```

Implementazione di Vector pt 1

```java
public class Vector<X>{

    private final static int INITIAL_SIZE = 10;

    private Object[] elements;  // Deposito elementi, non posso usare X[]!!
    private int size;   // Numero di elementi

    public Vector(){    // Inizialmente vuoto
      this.elements = new Object[INITIAL_SIZE];
      this.size = 0;
    }

    public void addElement(final X e){
      if (this.size == elements.length){
          this.expand();  // Se non c'è spazio
      }
      this.elements[this.size] = e;
      this.size++;
    }

    public X getElementAt(final int position){
      // unchecked warning
      return (X)this.elements[position];
    }
```

Implementazione di Vector pt 2

```
1
2     public int getLength(){
3       return this.size;
4     }
5
6     private void expand(){  // Raddoppio lo spazio..
7       final Object[] newElements = new Object[this.elements.length*2];
8       for (int i=0; i < this.elements.length; i++){
9           newElements[i] = this.elements[i];
10      }
11      this.elements = newElements;
12    }
13
14    public String toString(){
15      String s="|";
16      for (int i=0; i < size; i++){
17          s = s + this.elements[i] + "|";
18      }
19      return s;
20    }
21 }
```

Ancora sugli "unchecked warning" coi generici

```
public class ShowCast {

  public static void main(String[] args) {
    Vector<String> v = new Vector<>();
    v.addElement("a");
    v.addElement("b");
    v.addElement("c");
    Object o = v;
    Vector<Integer> v2 = (Vector<Integer>)o; // unchecked warning
    // Da qui in poi siamo in situazione "pericolosa"
    // Cosa può succedere?
    System.out.println(v2.getElementAt(0));
    System.out.println(v2.getElementAt(0).intValue());
  }
}
```

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

Polimorfismo vincolato

Negli esempi visti finora..

- Data una classe `C<X>`, `X` può essere istanziato a qualunque sottotipo di `Object`
- In effetti la definizione `class C<X>{..}` equivale a `class C<X extends Object>{..}`

Polimorfismo vincolato (Igarashi & Pierce & Wadler, 2000)

- In generale invece, può essere opportuno vincolare in qualche modo le possibili istanziazioni di `X`, ad essere sottotipo di un tipo più specifico di `Object`
- `class C<X extends D>{..}`
- In tal caso, dentro `C`, si potrà assumere che gli oggetti di tipo `X` rispondano ai metodi della classe `D`

LampsRow generica: Definizione

```java
public class LampsRow<L extends SimpleLamp> {

    private Vector<L> lamps;

    public LampsRow(){
      this.lamps = new Vector<>(); // inferenza
    }

    public L getLamp(int position){
      return this.lamps.getElementAt(position);
    }

    public void addLamp(L lamp){
      this.lamps.addElement(lamp);
    }

    public void switchOffAll(){
      for (int i=0;i<lamps.getLength();i++){
          this.getLamp(i).switchOff();
      }
    }

    public String toString(){
      return this.lamps.toString();
    }
}
```

Uso

Motivazione per questa genericità. Ha senso se:

- si ritiene molto frequente l'uso di `SimpleLamp` simili tra loro, ossia di una comune specializzazione (classe)
- è frequente l'uso di `getLamp()` e quindi del cast

```
public class UseLampsRow {

    public static void main(String[] s){
      final LampsRow<UnlimitedLamp> lr = new LampsRow<>();
      lr.addLamp(new UnlimitedLamp());
      lr.addLamp(new UnlimitedLamp());
      lr.addLamp(new UnlimitedLamp());

      lr.getLamp(0).switchOn();
      lr.switchOffAll();

      System.out.println(lr.getLamp(0).isOver());
      System.out.println(lr);
    }
}
```

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

Java Wildcards

```
// Gerarchia dei wrapper Numbers in java.lang
abstract class Number extends Object{...}
class Integer extends Number{...}
class Double extends Number{...}
class Long extends Number{...}
class Float extends Number{...}
...
```

```
// Accetta qualunque Vector<T> con T <: Number
// Vector<Integer>, Vector<Double>, Vector<Float>,...
void m(Vector<? extends Number> arg){...}

// Accetta qualunque Vector<T>
void m(Vector<?> arg){...}

// Accetta qualunque Vector<T> con Integer <: T
// Vector<Integer>, Vector<Number>, e Vector<Object> solo!
void m(Vector<? super Integer> arg){...}
```

Java Wildcards

3 tipi di wildcard (Igarashi & Viroli, 2002)

- Bounded (covariante): `C<? extends T>`
  - accetta un qualunque `C<S>` con `S <: T`
- Bounded (controvariante): `C<? super T>`
  - accetta un qualunque `C<S>` con `S >: T`
- Unbounded: `C<?>`
  - accetta un qualunque `C<S>`

Uso delle librerie che dichiarano tipi wildcard

- Piuttosto semplice, basta passare un argomento compatibile o si ha un errore a tempo di compilazione

Progettazione di librerie che usano tipi wildcard

- Molto avanzato: le wildcard pongono limiti alle operazioni che uno può eseguire, derivanti dal principio di sostituibilità

Approfondimento: sulla sostituibilità dei generici

Domanda: `Vector<Integer>` è un sottotipo di `Vector<Object>`?

Ovvero, possiamo pensare di passare un `Vector<Integer>` in tutti i contesti in cui invece ci si aspetta un `Vector<Object>`?

Risposta: no!! Sembrerebbe di si.. ma:

cosa succede se nel metodo qui sotto passiamo un `Vector<Integer>`?
⇒ potremmo facilmente compromettere l'integrità del vettore

```
void addAString(Vector<Object> vector){
    vector.addElement("warning!");
}
...
Vector<Integer> vec=new Vector<>(); // Inferenza
vec.addElement(new Integer(0));
vec.addElement(new Integer(1));
vec.addElement(new Integer(2));
addAString(vec);  // ATTENZIONE!!
int n = vec.getElementAt(3).intValue(); // break!
```

Subtyping e safety in OOP!

Safety di un linguaggio OO

Se nessuna combinazione di istruzioni porta a poter invocare un metodo su un oggetto la cui classe non lo definisce

- È necessario che il subtyping segua il principio di sostituibilità

Più in generale, se non possono accadere errori a tempo di esecuzione..

Java

- Si pone dove possibile l'obbiettivo della safety
- Quindi, non è vero che `Vector<Integer> <: Vector<Object>`

Generici e safety

In generale, istanziazioni diverse di una classe generica sono scollegate

- non c'è covarianza: non è vero che `C<T> <: C<S>` con `T <: S`
- non c'è controvarianza: non è vero che `C<S> <: C<T>` con `T <: S`

Unsafety con gli array di Java

Gli array di Java sono trattati come covarianti!

- Gli array assomigliano moltissimo ad un tipo generico
- `Integer[]` ∼ `Array<Integer>`, `T[]` ∼ `Array<T>`
- E quindi sappiamo che non sarebbe safe gestirli con covarianza
- E invece in Java è esattamente così!! P.e. `Integer[] <: Object[]`
- Quindi ogni scrittura su array potrebbe potenzialmente fallire....lanciando un `ArrayStoreException`

```
1 Object[] o = new Integer[]{1,2,3}; // OK per covarianza
2 o[0] = "a"; // Lancia ArrayStoreException
3 // per prevenire la unsafety di successive istruzioni come:
4 // Integer[] ar=((Integer[])o);
5 // int i=ar[0].intValue();
```

Covarianza e operazioni di accesso

La covarianza (`C<T> <: C<S>` con `T <: S`) sarebbe ammissibile se:

- La classe `C<X>` non avesse operazioni che ricevono oggetti `X`
- Ossia, ha solo campi privati e nessun metodo con argomento `X`

La controvarianza (`C<S> <: C<T>` con `T <: S`) sarebbe ammissibile se:

- La classe `C<X>` non avesse operazioni che producono oggetti `X`
- Ossia, ha solo campi privati e nessun metodo con tipo di ritorno `X`

In pratica:

- La maggior parte delle classi generiche `C<X>` hanno campi di tipo `X` (composizione) e operazioni getter e setter, e quindi per loro covarianza e controvarianza non funzionano
- Con le wildcard si aumenta il riuso dei metodi che non usano tutte le funzione del tipo generico in input

Esempio Wildcard Bounded covariante: C<? extends T>

```java
public class Cov{

    public static void printAll(Vector<? extends Number> vect){
      // Questo metodo usa solo 'X getElementAt()'
      // Quindi l'input può essere covariante!
      // Si noti che Number definisce 'int intValue()'
      for (int i=0;i<vect.getLength();i++){
          System.out.println(vect.getElementAt(i).intValue());
      }
    }

    public static void main(String[] s){
      Vector<Integer> vector = new Vector<>();
      vector.addElement(1);
      vector.addElement(2);
      vector.addElement(3);
      // Posso passare Vector<Integer> dove si attende
      // Vector<? extends Number>
      printAll(vector);
    }
}
```

Esempio Wildcard Bounded controvariante: C<? super T>

```
public class Contra{

    public static void addStrings(Vector<? super String> vect,
    String s, int n){
      // Questo metodo usa solo 'void addElement(X x)'
      // Quindi l'input può essere contra-variante!
      for (int i=0;i<n;i++){
          vect.addElement(s);
      }
    }

    public static void main(String[] s){
      Vector<Object> vector = new Vector<>();
      vector.addElement(1);
      vector.addElement(new java.util.Date());
      // Posso passare Vector<Object> dove si attende
      // Vector<? super String>
      addStrings(vector,"aggiunta",10);
      System.out.println(vector);
    }
}
```

Esempio Wildcard Unbounded: C<?>

```
public class Unbounded{

    public static void printLength(Vector<?> vect){
      // Questo metodo usa solo 'int getLength()'
      // Quindi l'input può essere bi-variante!
      System.out.println(vect.getLength());
    }

    public static void main(String[] s){
      Vector<Integer> vector = new Vector<>();
      vector.addElement(1);
      vector.addElement(2);
      vector.addElement(3);
      // Posso passare Vector<Integer> dove si attende
      // Vector<?>
      printLength(vector);
    }
}
```

Un uso tipico: metodi in classi generiche

```
public class Vector<X>{
    ...
    public void addAllFrom(Vector<? extends X> vect){
      for (int i=0;i<vect.getLength();i++){
          this.addElement(vect.getElementAt(i));
      }
    }
}
```

```
public class UseVector{
    public static void main(String[] s){
      Vector<Object> vo = new Vector<>();
      vo.addElement(1);
      vo.addElement("2");
      vo.addElement(new java.util.Date());

      Vector<Double> vd = new Vector<>();
      vd.addElement(Math.random());
      vd.addElement(Math.random());

      vo.addAll(vd);
      System.out.println(vo);
    }
}
```

Un uso tipico: metodi in classi generiche

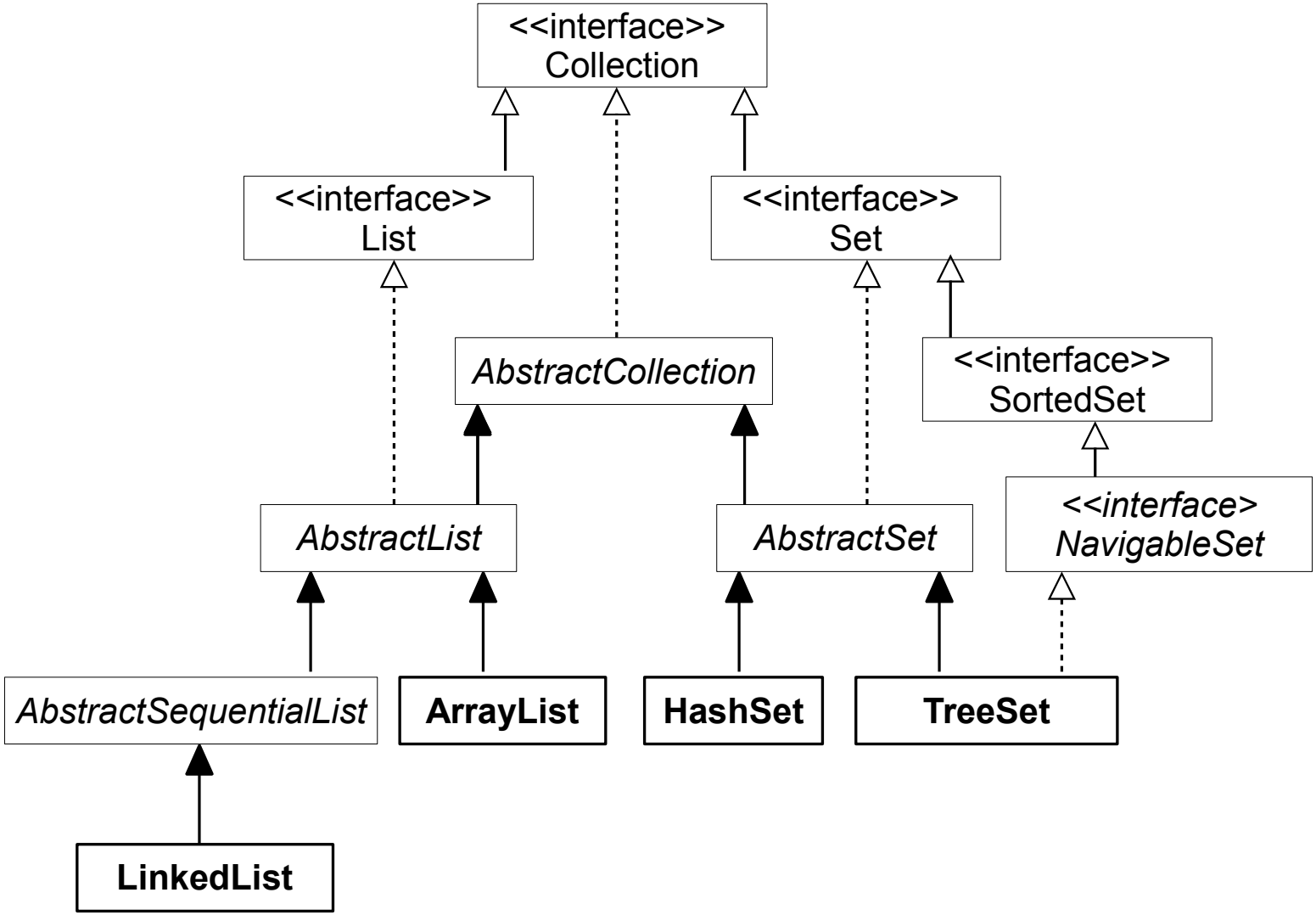
```java
public class Vector<X>{
    ...
    public void addAllFrom(Vector<? extends X> vect){
      for (int i=0;i<vect.getLength();i++){
          this.addElement(vect.getElementAt(i));
      }
    }
}
```

```java
public class UseVector{
    public static void main(String[] s){
      Vector<Object> vo = new Vector<>();
      vo.addElement(1);
      vo.addElement("2");
      vo.addElement(new java.util.Date());

      Vector<Double> vd = new Vector<>();
      vd.addElement(Math.random());
      vd.addElement(Math.random());

      vo.addAll(vd);
      System.out.println(vo);
    }
}
```

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

Implementazione collezioni – UML



List

```java
public interface List<E> extends Collection<E> {
    // Additional Bulk Operations
    // aggiunge gli elementi in pos. index
    boolean addAll(int index, Collection<? extends E> c);

    // Positional Access Operations
    E get(int index);
    E set(int index, E element);
    void add(int index, E element);
    E remove(int index);

    // Search Operations
    int indexOf(Object o);  // basato su Object.equals
    int lastIndexOf(Object o);  // basato su Object.equals

    // List Iterators
    ListIterator<E> listIterator();
    ListIterator<E> listIterator(int index);

    // View
    List<E> subList(int fromIndex, int toIndex);
}
```

Implementazioni di `List`

Caratteristiche delle liste

- Sono sequenze: ogni elemento ha una posizione

→ Il problema fondamentale è realizzare i metodi posizionali in modo efficiente, considerando il fatto che la lista può modificarsi nel tempo (altrimenti andrebbe bene un array)

Approccio 1: `ArrayList`

Internamente usa un array di elementi con capacità maggiore di quella al momento necessaria. Se serve ulteriore spazio si alloca trasparentemente un nuovo e più grande array

Approccio 2: `LinkedList`

Usa una double-linked list. L'oggetto `LinkedList` mantiene il riferimento al primo e ultimo elemento della lista, e alla dimensione della lista

ArrayList

Caratteristiche di performance

- Lettura/scrittura in data posizione sono a tempo costante
- La `add()` è tempo costante ammortizzato, ossia, n add si effettuano in $O(n)$
- Tutte le altre operazioni sono a tempo lineare

Funzionalità aggiuntive

Per migliorare le performance (e l'occupazione in memoria) in taluni casi l'utente esperto può usare funzioni aggiuntive

- Specificare la dim iniziale dell'array interno nella `new`
- `trimToSize()` e `ensureCapacity()` per modifiche in itinere

ArrayList: aspetti aggiuntivi

```
public class ArrayList<E> extends AbstractList<E>
        implements List<E>, RandomAccess, Cloneable, java.io.
    Serializable
{
    public ArrayList(int initialCapacity) {...}

    // Usa un valore di default per la capacità iniziale (10)
    public ArrayList() {...}

    // Riempe coi valori di c
    public ArrayList(Collection<? extends E> c) {...}

    // Riduce la dimensione dell'array interno
    public void trimToSize() {...}

    // Aumenta la dimensione dell'array interno
    public void ensureCapacity(int minCapacity) {...}

}
```

UseArrayList

```java
public class UseArrayList{

  public static void main(String[] s){

    final ArrayList<Persona> alist = new ArrayList<>();
      alist.ensureCapacity(30); // per performance
      for (int anno=1960;anno<1970;anno++){
          alist.add(new Persona("Rossi",anno,false));
          alist.add(new Persona("Bianchi",anno,true));
          alist.add(new Persona("Verdi",anno,false));
      }
      final Persona p = new Persona("Rossi",1967,false);
      int pos = alist.indexOf(p);
      System.out.println(p+" in position "+pos);

      // Iteratore da pos fino in fondo.. lo uso per eliminare
      final ListIterator<Persona> iterator = alist.listIterator(pos);
      while (iterator.hasNext()){
          iterator.next();
          iterator.remove();
      }
      for (final Persona p2: alist){
          System.out.println(alist.indexOf(p2)+"\t"+p2);
      }
      alist.trimToSize(); // riduco le dimensioni
    }
}
```

LinkedList

Caratteristiche di performance

- Accesso e modifica in una data posizione hanno costo lineare
- Operazioni in testa o coda, quindi, sono a tempo costante
- Usa in generale meglio la memoria rispetto `ArrayList`
- (di norma però si preferisce `ArrayList`)

Caratteristiche aggiuntive

- Implementa anche l'interfaccia `Queue`, che modella una coda (ad esempio FIFO)
- Implementa anche l'interfaccia `Deque` che estende `Queue`, una coda a due estremi

LinkedList: funzioni aggiuntive relative a code (e stack)

```
1  public interface Queue<E> extends Collection<E> {
2      boolean offer(E e); // inserisce se c'è spazio
3      E poll();    // preleva se c'è il primo
4      E element();         // legge se c'è il primo, altrimenti eccezione
5      E peek();    // legge se c'è il primo, altrimenti null
6  }
```

```
1  public interface Deque<E> extends Queue<E> {
2      void addFirst(E e);
3      void addLast(E e);
4      boolean offerFirst(E e);
5      boolean offerLast(E e);
6      E removeFirst();
7      E removeLast();
8      E pollFirst();
9      E pollLast();
10     E getFirst();
11     E getLast();
12     E peekFirst();
13     E peekLast();
14     boolean removeFirstOccurrence(Object o);
15     boolean removeLastOccurrence(Object o);
16     void push(E e); // identical to addFirst()
17     E pop();        // identical to removeFirst()
18 }
```

LinkedList: costruzione

```
1 public class LinkedList<E>
2     extends AbstractSequentialList<E>
3     implements List<E>, Deque<E>, Cloneable, java.io.Serializable {
4
5     public LinkedList() {...}
6
7     public LinkedList(Collection<? extends E> c) {...}
8 }
```

UseLinkedList

```java
public class UseLinkedList{

  private static final String ELEMS = "A B C D E F G H I L M";

    public static void main(String[] s){
      final LinkedList<String> llist =
        new LinkedList<>(Arrays.asList(ELEMS.split(" ")));
      llist.addFirst("*");
      llist.addLast("*");
      llist.removeFirstOccurrence("*");
      llist.removeLastOccurrence("*");

      // bad performance
      llist.add(llist.indexOf("L"),"K");
      // better performance
      final ListIterator<String> it = llist.listIterator();
      while (it.hasNext()){
          if (it.next().equals("I")){
            it.add("J");
            break;
          }
      }
      final String[] str = llist.toArray(new String[0]);
      System.out.println(Arrays.toString(str));
    }
}
```

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

Classi di utilità (moduli): Arrays e Collections

java.util.Arrays

- Contiene varie funzionalità d'ausilio alla gestione degli array
- In genere ha varie versione dei metodi per ogni array di tipo primitivo
- Ricerca binaria (dicotomica), Ordinamento (quicksort), copia
- Operazioni base (`toString`, `equals`, `hashCode`), anche ricorsive

java.util.Collections

- Raccoglie metodi statici che sarebbero potuti appartenere alle varie classi/interfacce viste
- Ricerca binaria (dicotomica), Ordinamento (quicksort), copia, min, max, sublist, replace, reverse, rotate, shuffle
- Con esempi notevoli d'uso delle wilcard

Arrays: qualche esempio di metodi

```
1
2  public class Arrays {
3      public static void sort(Object[] a, int fromIndex, int toIndex) {...}
4      public static <T> void sort(T[] a, Comparator<? super T> c) {...}
5      ...
6      public static int binarySearch(int[] a, int key) {...}
7      public static int binarySearch(char[] a, char key) {...}
8      public static <T> int binarySearch(T[] a, T key, Comparator<? super T> c
       ) {...}
9      ...
10     public static <T> List<T> asList(T... a) {...}
11     public static <T> T[] copyOfRange(T[] original, int from, int to) {...}
12     public static void fill(Object[] a, int from, int to, Object val) {...}
13
14     public static boolean deepEquals(Object[] a1, Object[] a2) {...}
15     public static int deepHashCode(Object a[]) {...}
16     public static String deepToString(Object[] a) {...}
17
18     public static String toString(long[] a) {...}
19     public static String toString(int[] a) {...}
20     public static String toString(Object[] a) {...}
21
22 }
```

UseArrays: qualche esempio di applicazione

```java
public class UseArrays implements Comparator<Integer>{

    public int compare(Integer a,Integer b){
      return b-a; // ordine inverso
    }

    public static void main(String[] s){

      String[] array = new String[] {"a","b","c","d","e","f"};

      List<String> list = Arrays.asList("a","b","c","d","e","f");

      final Integer[] a = new Integer[20];
      for (int i=0;i<20;i++){
          a[i] = (int)(Math.random()*100);
      }
      System.out.println("rand: "+Arrays.toString(a));
      Arrays.sort(a); // sort in ordine naturale
      System.out.println("sort1: "+Arrays.toString(a));
      Arrays.sort(a,new UseArrays()); // sort col comparator
      System.out.println("sort2: "+Arrays.toString(a));
      Arrays.fill(a,10,15,0); // fill nel range
      System.out.println("fill: "+Arrays.toString(a));

      final Integer[][] b = new Integer[10][];
      Arrays.fill(b,a); // fill di un array di array
      System.out.println("recu: "+Arrays.deepToString(b));
    }
```

Collections: qualche esempio di metodi

```
// nota: tutti metodi public static!!
public class Collections {
    // ordinamenti e varie
    <T extends Comparable<? super T>> void sort(List<T> list) {...}
    <T> void sort(List<T> list, Comparator<? super T> c) {...}
    <T> int binarySearch(List<? extends Comparable<? super T>> list, T key)
    <T> T min(Collection<? extends T> coll, Comparator<? super T> comp)

    // modifiche
    void reverse(List<?> list) {...}
    void shuffle(List<?> list) {...}
    <T> void fill(List<? super T> list, T obj) {...}
    <T> void copy(List<? super T> dest, List<? extends T> src) {...}

    // letture varie
    int indexOfSubList(List<?> source, List<?> target) {...}
    boolean disjoint(Collection<?> c1, Collection<?> c2) {...}
    int frequency(Collection<?> c, Object o) {...}

    // costruzioni di collezioni
    <T> List<T> emptyList() {...}
    <T> Set<T> emptySet() {...}
    <T> List<T> nCopies(int n, T o) {...}
    <T> Set<T> singleton(T o) {...}
    <T> List<T> singletonList(T o) {...}
}
```

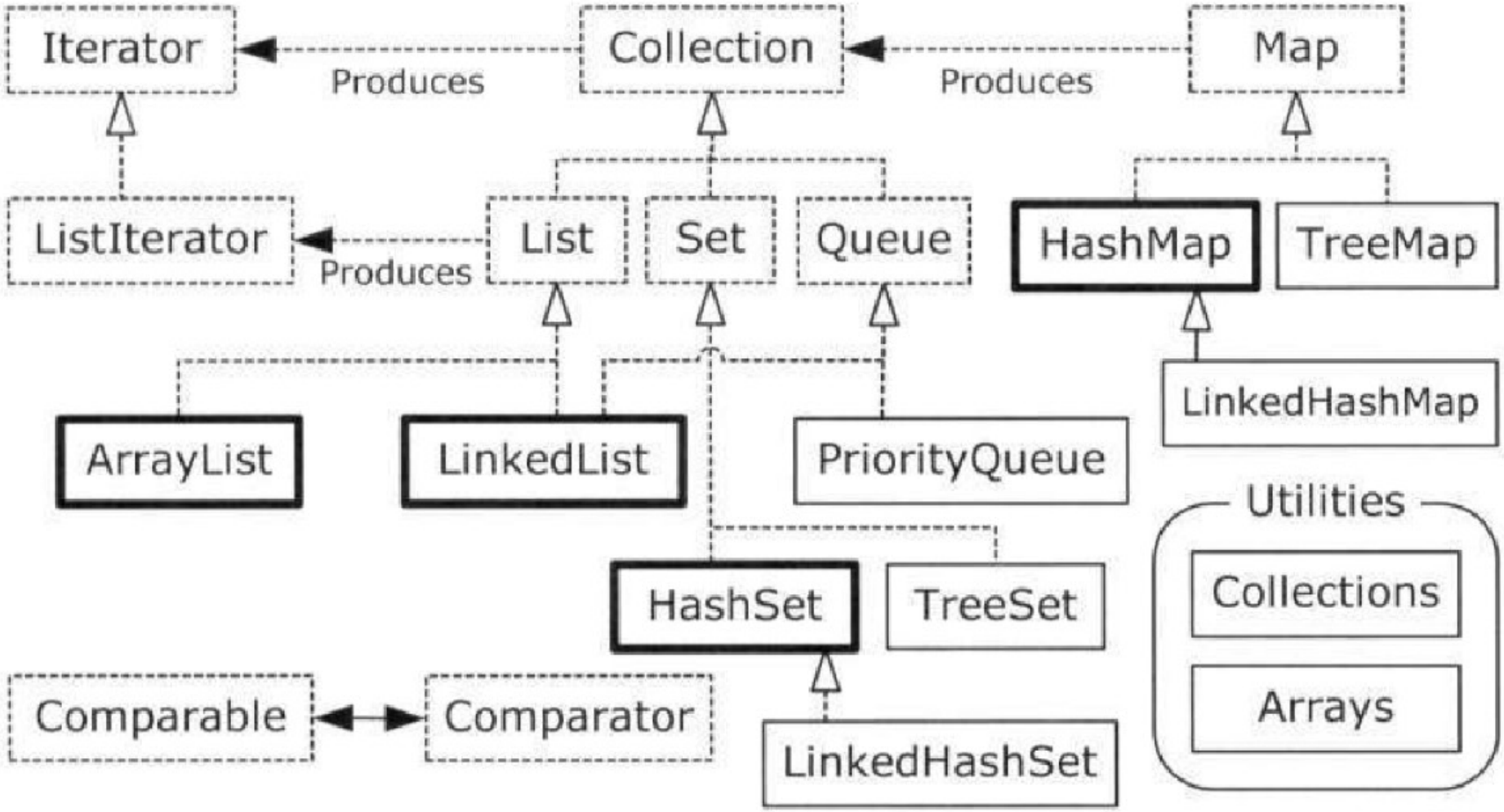
UseCollections: qualche esempio di applicazione

```
public class UseCollections{

    public static void main(String[] s){
      final List<Integer> list = Arrays.asList(new Integer[]{0,1,2,3,4});
      final Set<List<Integer>> set = new HashSet<>();

      for (int i=0;i<5;i++){
        final List<Integer> l2 = new ArrayList<>(list);
          Collections.shuffle(l2);// permuto
          if (!set.contains(l2)){ // no duplicazione!
            set.add(l2);      // aggiungo
          }
      }
      System.out.println("shuf: "+set); //[[4,1,2,3,0],[3,1,4,0,2],..

      int ct=0;
      for (final List<Integer> l:set){
          Collections.fill(l,ct++);
      }
      System.out.println("inc: "+set); //[[0,0,0,0,0],[1,1,1,1,1],..

      System.out.println("cop: "+Collections.nCopies(5,list));
      //[[0,1,2,3,4],[0,1,2,3,4],..
    }
}
```

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

JCF – struttura semplificata



Map

```
public interface Map<K,V> {

    // Query Operations
    int size();
    boolean isEmpty();
    boolean containsKey(Object key);          // usa Object.equals
    boolean containsValue(Object value);      // usa Object.equals
    V get(Object key);                        // accesso a valore

    // Modification Operations
    V put(K key, V value);            // inserimento chiave-valore
    V remove(Object key);             // rimozione chiave(-valore)

    // Bulk Operations
    void putAll(Map<? extends K, ? extends V> m);
    void clear();                     // cancella tutti

    // Views
    Set<K> keySet();                      // set di valori
    Collection<V> values();               // collezione di chiavi

    ... // qualche altro
}
```

Usare le mappe

```java
public class UseMap {

  public static void main(String[] args) {
    // Uso una incarnazione, ma poi lavoro sull'interfaccia
    final Map<Integer, String> map = new HashMap<>();
    // Una mappa è una funzione discreta
    map.put(345211, "Bianchi");
    map.put(345122, "Rossi");
    map.put(243001, "Verdi");
    System.out.println(map); // {345211=Bianchi, 243001=Verdi, 345122=Rossi}
    map.put(243001, "Neri"); // Rimpiazza Verdi

    // modo poco prestante per accedere alle coppie chiave-valore
    for (final Integer i : map.keySet()) {
      System.out.println("Chiave: " + i + " Valore: " + map.get(i));
    }
    // modo prestante per accedere ai soli valori
    for (final String s : map.values()) {
      System.out.println("Valore: " + s);
    }
  }
}
```

Due implementazioni di Map e AbstractMap

Map<K,V>

- Rappresenta una funzione dal dominio K in V
- La mappa tiene tutte le associazioni (o "entry")
- Non posso esistere due entry con stessa chiave (Object.equals)

HashMap

- Sostanzialmente un HashSet di coppie Key, Value
- L'accesso ad un valore tramite la chiave è fatto con hashing
- Accesso a tempo costante, a discapito di overhead in memoria

TreeMap

- Sostanzialmente un TreeSet di coppie Key, Value
- L'accesso ad un valore tramite la chiave è fatto con red-black tree
- Accesso in tempo logaritmico
- Le chiavi devono essere ordinate, come per i TreeSet

Outline

1. Il problema della type-erasure
2. Polimorfismo vincolato
3. Java Wildcards e sostituibilità
4. Implementazioni di `List`
5. Altre classi: `Arrays` e `Collections`
6. Il caso delle `java.util.Map`
7. Un esercizio sulle collezioni

Interfaccia da implementare

```
import java.util.*;

public interface Graph<N> {

  // Adds a node: nothing happens if node is null or already there
  void addNode(N node);

  // Adds an edge: nothing happens if source or target are null
  void addEdge(N source, N target);

  // Returns all the nodes
  Set<N> nodeSet();

  // Returns all the nodes directly targeted from node
  Set<N> linkedNodes(N node);

  // Gets one sequence of nodes connecting source to path
  List<N> getPath(N source, N target);

}
```

Codice di prova

```
public class UseGraph{

public static void main(String[] args){
  Graph<String> g = null; //new GraphImpl<>();

  g.addNode("a");
  g.addNode("b");
  g.addNode("c");
  g.addNode("d");
  g.addNode("e");

  g.addEdge("a","b");
  g.addEdge("b","c");
  g.addEdge("c","d");
  g.addEdge("d","e");
  g.addEdge("c","a");

  System.out.println(g.nodeSet());
  // ["a","b","c","d","e"].. in any order
  System.out.println(g.linkedNodes("c"));
  // ["d","a"].. in any order
  System.out.println(g.getPath("b","a"));
  // either the path b,c,a or b,c,d,e,a


  }
}
```

Strategia risolutiva

Passi:

1. Capire bene cosa la classe deve realizzare
2. Pensare a quale tipo di collezioni può risolvere il problema in modo semplice e prestante
3. Realizzare i vari metodi
4. Controllare i casi particolari